

Programação Assíncrona

Objetivo da Aula

Compreender os princípios de programação assíncrona, no contexto do JavaScript, e aplicar em situações práticas comuns para sistemas desenvolvidos na plataforma NodeJS.

Apresentação

O processamento concorrente de tarefas pelo processador trouxe grandes possibilidades para o mundo da computação, como nos jogos de ação, em que o uso de threads é uma necessidade inerente ao tipo de aplicativo. Nos servidores da internet, com múltiplas conexões simultâneas, o uso de threads também é um elemento fundamental para viabilizar diversas funcionalidades.

Na plataforma NodeJS, a preocupação com o paralelismo é muito forte, pois a própria adoção do modelo single thread já evidencia o objetivo de trabalhar com modelos assíncronos. Essa é uma grande preocupação nas operações de entrada e saída, já que são muito demoradas, podendo bloquear algum processo e, até mesmo, toda a plataforma.

Como a sintaxe JavaScript já oferece diversos ferramentais para lidar com a programação assíncrona, devemos compreender esses elementos e saber como utilizá-los, junto às bibliotecas do NodeJS, para criar sistemas eficientes e que não estejam sujeitos a bloqueios durante a execução.

1. Elementos para Programação Assíncrona

Quando você trabalha com um processo síncrono, a execução segue uma sequência finita de passos, em que ocorre o bloqueio na chamada de sub-rotinas, o que expressa muito bem os algoritmos tradicionais. Porém, alguns processos podem requerer a execução de tarefas em paralelo, necessitando de uma nova abordagem, que leve em conta o paralelismo e a sincronização de tarefas.

Ao nível do sistema operacional, um thread representa um trecho de código que pode ser executado no processador por um período máximo, denominado “time slice”, ou até que ocorra a solicitação de uma operação de entrada ou saída para algum dispositivo. Nas duas situações, a execução é paralisada, o próximo thread da fila de execução ocupa o processador, e o thread interrompido volta para a fila, caracterizando o comportamento multitarefa.

Cada aplicativo é representado por um processo, o qual tem pelo menos um thread, fazendo com que todos os programas executem, de forma concorrente, com a mesma probabilidade de uso do processador. Porém, o aplicativo pode criar outros threads, fazendo com que o processador seja utilizado mais vezes para o mesmo processo, o que acaba representando um aumento de eficiência.

Quando utilizamos uma linguagem qualquer, seus componentes de software podem encapsular as chamadas para a API do sistema operacional, de forma a tirar proveito do modelo concorrente. Isso fica evidente na linguagem C, como pode ser observado no fragmento de código seguinte, no qual **pthread_create** cria e inicia um novo thread, enquanto **pthread_join** aguarda a finalização do thread.

```
#include <stdio.h>
#include <pthread.h>
void *func(void *args) {
    printf("\nOLA\n");
    return (NULL);
}
int main () {
    pthread_t threads[5];
    int i;
    for(i=0;i<5;i++)
        pthread_create(&threads[i], NULL, func, NULL);
    for(i=0;i<5;i++)
        pthread_join(threads[i], NULL);
    return 0;
}
```

No modelo assíncrono, a função executa em um novo thread, sem o bloqueio do fluxo principal, exigindo métodos de sincronização quando há dependência do resultado dessa execução. Ao trabalhar com a linguagem JavaScript, usamos a palavra reservada **async** para configurar uma função como assíncrona, além de sincronizarmos a execução com **await** e **then**.

O retorno de uma função assíncrona é sempre do tipo **promise**, logo, se uma função comum tem como retorno um promise, ela se comporta da mesma forma que uma função assinada com **async**. Ao configurar um retorno como promise, você não fornecerá o dado em si, mas a possibilidade de existência do valor, em algum instante futuro, podendo ocorrer uma exceção antes do retorno.

Na listagem seguinte, podemos observar o uso de **async** na implementação da clássica série de Fibonacci.

```
const { promisify } = require('util');
const sleep = promisify(setTimeout);

const fibo = async(n) => {
  let x1 = 1, x2 = 1;
  for(let a=2; a<=n; a++){
    let c = x1 + x2;
    x1 = x2; x2 = c;
    await sleep(10);
  }
  return x2;
}

const imprimir = async() => {
  for(let i = 19; i >= 0; i--){
    const f = await fibo(i);
    console.log(`Fibo(${i}) = ${f}`);
  }
}

imprimir();
```

Executando o código, teríamos a impressão dos valores da série, de forma **decrecente**, para as posições 19 até 0. Como o uso de **await** é bloqueante, a ordem de execução é respeitada.



Destaque

O uso de **await** é permitido apenas em funções marcadas como **async**. Isso é justificado pelo fato de que bloqueia o thread corrente; ainda, se fosse utilizado no fluxo síncrono principal, iria bloquear o aplicativo.

Também foi utilizado o **promisify**, da biblioteca **util**, para permitir a chamada de **setTimeout** no modelo assíncrono. A nova função, que chamamos de **sleep**, irá permitir pausar o thread corrente por um prazo definido em milissegundos.

Opcionalmente, você poderá capturar o retorno da função assíncrona através do operador **then**. A diferença, em relação ao **await**, é que ele não bloqueia a execução, sendo possível utilizar no fluxo síncrono, e exige uma função de **callback** para capturar o valor de retorno.

A função imprimir pode ser reescrita para uso de **then**, como é apresentado na listagem seguinte.

```
const imprimir = () => {
  for(let i = 19; i >= 0; i--){
    fibo(i).then((f)=>console.log(`Fibo(${i}) = ${f}`));
  }
}
```

Executando novamente, teremos como resultado a impressão dos valores na ordem **crecente**. Isso ocorre porque o uso de **sleep**, no loop de cálculo para o valor da série, fará com que as chamadas iniciais sejam mais demoradas do que as finais, e, como a execução é paralela, elas irão terminar depois.

Outra forma de escrever uma função assíncrona é através de um retorno do tipo **promise**, como no exemplo seguinte.

```
const dividir = (a,b) => {  
  const promise = new Promise((resolve, reject)=>{  
    if(b==0){ reject("Divisao por zero"); }  
    else { resolve(a/b); }  
  });  
  return promise;  
}
```

Ao definir um retorno do tipo promise, o elemento **resolve** retorna com o valor da operação para o chamador, enquanto **reject** causa uma exceção. A captura da exceção ocorreria via operador **catch** em uma chamada com **then**.

```
for(let i = 2; i>=-2; i--)  
  dividir(10,i).then((x)=>console.log(x))  
    .catch((error)=>console.log(error));
```

Se quiser chamar a função de divisão, a partir de uma função marcada como **async**, você precisará utilizar **await**, tratando a ocorrência de exceção em um bloco com **try** e **catch**.

```
const exec = async () => {  
  for(let i = 2; i>=-2; i--)  
    try {  
      console.log(await dividir(10,i));  
    }catch(error){  
      console.log(error);  
    }  
}
```

Finalmente, se quiser gerar a exceção em uma função marcada como **async**, você precisará lançá-la através de uma cláusula **throw**.

```
const dividir2 = async(a,b) => {
  if(b==0)
    throw("Divisao por zero");
  return a/b;
}
```

2. Tratamento Assíncrono de Arquivos

Toda operação de leitura ou escrita em dispositivos é demorada, quando a comparamos com as operações de execução direta no processador, e por isso ocorre a divisão entre instruções **I/O Bound** e **CPU Bound**. Se o processador tiver de aguardar a finalização de uma operação de leitura ou escrita, ele ficará com grande ociosidade, ao compararmos sua velocidade com a dos dispositivos.

Na prática, em sistemas operacionais multitarefas, temos diversos programas executando de forma concorrente, e cada tarefa tem um prazo para ocupar a CPU, denominado "time slice". Porém, se uma tarefa solicita operação de leitura ou escrita, ela é delegada para o controlador de dispositivos, e a tarefa é substituída antes do final do time slice, além de voltar para a fila de execução apenas após o término da operação.

Se o seu programa faz a leitura de arquivo de forma síncrona, o processador irá liberar a execução da próxima linha de código apenas ao final da leitura, o que levará ao bloqueio do restante da execução. O mesmo problema ocorre nas operações de escrita, que também são delegadas pelo processador para os controladores de dispositivos.

A solução mais simples para que não ocorra o bloqueio da execução é o uso de métodos assíncronos, nos quais a leitura ou escrita fica sob a responsabilidade de uma nova tarefa, enquanto o fluxo de execução principal segue sem bloqueios. Claro que o tratamento da informação irá exigir meios de sincronização, já que os dados não estarão disponíveis de imediato, sendo retornados via **promises**.



Destaque

Alguns dos exemplos utilizarão a palavra **import**, exigindo o atributo **type**, com valor **module**, no arquivo **package.json**. Com a configuração adotada, um arquivo só permitirá o uso de **require** se tiver a extensão **cjs**.

Nossa primeira operação com arquivos será a escrita assíncrona, de acordo com a listagem apresentada a seguir.

```
const $fs = require("fs");

$fs.appendFile('log.txt', `Escrito em ${new Date()}\n`, 'utf8',
  (err)=>{
    if(err) throw err;
    console.log("Linha adicionada");
  });
```

O método **appendFile** utiliza o nome do arquivo, o conteúdo que será escrito, a codificação e uma callback que recebe o código do erro, caso este ocorra. Esse método irá adicionar informação ao arquivo, de forma assíncrona, além de criar o arquivo quando necessário.

Caso você precise escrever diversas linhas no arquivo, será melhor mantê-lo aberto e fechar apenas após a última linha. Nesse contexto, devemos utilizar um descritor de arquivo, com os métodos **open** e **close**.

```
import {open,close,appendFile} from "fs"

const closeArq = (fd) => {
  close(fd,(err)=>{ if (err) throw err; });
}

open('log.txt', 'a', (err,fd) =>{
  if (err) throw err;
  appendFile(fd, `Escrito X em ${new Date()}\n`, 'utf8',
    (err) => { closeArq(fd); if (err) throw err; });
});
```

Além da utilização através de callbacks, é possível trabalhar com chamadas sincronizadas, por meio de `await`, sendo utilizada a biblioteca **fs/promises**, como pode ser observado a seguir.

```
const $fs = require('fs/promises');
async function teste_leitura() {
  try {
    const data = await $fs.readFile(
      'log.txt', {encoding: 'utf8'});
    console.log(data);
  } catch (err) { console.log(err); }
}
teste_leitura();
```

O método **readFile** efetua a leitura completa do arquivo, recebendo o nome do arquivo e a codificação utilizada. Como foi utilizado `await`, o conteúdo completo do arquivo é recebido na variável `data`, sendo impresso, em seguida, no console.

3. Comunicação em Rede Assíncrona

Da mesma forma que a manipulação de arquivos exige muito tempo para a execução, a comunicação em rede também terá esse perfil. Mais que isso, temos diversos outros fatores que podem levar a uma demora maior na comunicação em rede, como a velocidade e a qualidade de transmissão.

Fica fácil observarmos que o uso de elementos assíncronos será necessário, também, nesse contexto, e as bibliotecas **net** e **http**, que fazem parte do núcleo do NodeJS, oferecem comportamento assíncrono de forma simples. Em geral, para a definição de **aplicativos web**, iremos utilizar a biblioteca **http**, mas, para comunicações de baixo nível, envolvendo a especificação de protocolos e o uso de **sockets**, adotaremos a biblioteca **net**.

Inicialmente, vamos criar um servidor com a biblioteca **net**.

```
const net = require('net');

const server = net.createServer((socket) => {
  socket.end(`Hora no servidor: ${new Date()}\n`);
}).on('error', (err) => {
  throw err;
});
server.listen(1234);
```

Na comunicação que segue esse modelo, o socket servidor não tem a função de transmitir dados, e sim apenas de escutar a rede à espera de pedidos de conexão. Quando um socket do lado cliente solicita a conexão, o socket servidor recebe o pedido, instancia um socket do lado servidor, delega a comunicação com aquele cliente específico para o novo socket e volta a escutar a rede.

Observando o código, você verá que a criação do socket servidor ocorre através de **createServer**, o qual utiliza uma callback que tem como parâmetro o **socket** gerado pelo servidor para se comunicar com o socket do cliente. Por se tratar de um protocolo muito simples, o socket do lado servidor apenas emite a mensagem, com a hora corrente, para o socket cliente, terminando a comunicação em seguida, através do método **end**.

No caso de erro ao iniciar o servidor, o que pode ocorrer pelo uso de uma porta já ocupada por outro serviço, ocorrerá o evento **error**, que irá executar uma callback para recepção e tratamento do erro. Aqui, nós apenas ecoamos o erro para a plataforma através de uma instrução **throw**.

Você pode observar a ativação do serviço na última linha, com a chamada para o método **listen**, utilizando a porta **1234**. É justamente nesse ponto que o socket servidor começa a escutar a rede.

Agora, precisamos definir o lado cliente, conforme o código seguinte, em que são criadas cinco conexões com o servidor anterior.

```
const net = require('net');

for(let i=1; i<=5; i++){
  const client = net.createConnection(1234, "localhost");
  client.on('data', (dados) => {
    console.log(dados.toString()); // classe Buffer
  })
  .on('end', () => {client.destroy();});
}
```

Para a criação do socket cliente, você deve utilizar **createConnection**, tendo como parâmetros a **porta** e o **endereço** do servidor. A conexão é estabelecida com a própria criação do socket, e a recepção de dados é tratada por meio do sistema de eventos.

É necessário associar ao evento **data** uma função callback para o tratamento dos dados enviados pelo servidor. Por se tratar de uma comunicação que pode envolver diversos tipos de dados, a recepção ocorre em um **buffer de bytes**, o qual pode ser convertido para texto, usando o método **toString**.

Ao final da comunicação, quando é recebido o evento **end**, o cliente deve ser destruído, como pode ser observado na callback do exemplo.



Destaque

Nosso exemplo é um time server simples, no qual a comunicação envolve apenas uma mensagem, mas as funções callback dos sockets, no cliente e no servidor, poderiam implementar qualquer protocolo mais complexo.

Embora seja comum ver a utilização de sockets em jogos e servidores para conteúdo binário, a maior parte dos sistemas comerciais executa através do protocolo HTTP. Uma prova de que a plataforma web necessita de elementos assíncronos é o modelo AJAX, no qual o XMLHttpRequest é utilizado em páginas HTML para carregar o conteúdo dinamicamente e de forma assíncrona – ou seja, uma oposição direta ao modelo clássico, em que a página era exibida apenas quando fosse carregada por completo.

A biblioteca **http** permite criar tanto servidores quanto requisições, utilizando elementos assíncronos em ambos os lados. Note que não falamos de cliente, mas de **requisição**, pois a comunicação envolve apenas uma chamada e uma resposta, e não uma conexão duradoura.

A implementação de um servidor HTTP pode ser observada a seguir.

```
const http = require('http');
const server = http.createServer((req, res) => {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'text/plain');
    res.end('Olá Mundo!');
});
server.listen(3000);
```

De forma semelhante ao socket servidor, o servidor HTTP tem uma callback para tratar qualquer requisição recebida, delegando para ela esse trabalho, ao mesmo tempo que continua escutando a rede, à espera de uma próxima requisição. Na função de callback, temos os parâmetros **req** e **res**, para receber a requisição e a resposta na comunicação HTTP com um cliente específico.

Você poderia testar o método HTTP utilizado ou obter os parâmetros, a partir de req, mas, em nosso exemplo, temos apenas a emissão de uma resposta com status 200, indicando sucesso, e um conteúdo em texto plano. Note que o tipo de conteúdo é especificado através de **setHeader**, sendo útil para o navegador escolher o visualizador correto, de acordo com o formato.

Através do método **listen**, configurado para a porta **3000**, o servidor começa a escutar a rede, passando a aceitar requisições.

Ao nível do aplicativo cliente, você precisa criar uma requisição, através do método **request** de **http**, informando o endereço, a porta e o método HTTP que será utilizado. Também é necessário definir o tratamento da resposta (**res**), e, aqui, é associado o evento **data**, que ocorre na recepção de dados, a uma função de callback para impressão no console.

```
const http = require('http');

const req = http.request(
  {hostname: 'localhost', port: 3000, method: 'GET'},
  (res) => {
    res.on('data', (d) => { console.log(d.toString()); });
  });
req.end();
```

Algo que fica claro em nossos exemplos é a combinação de diversos modelos de programação assíncrona sendo utilizada na comunicação em rede, tanto no lado cliente quanto no servidor.

4. Eventos e Tratamento Assíncrono

Uma peça fundamental na arquitetura do NodeJS é o **event loop**, e não é à toa que muitos processos assíncronos são iniciados por eventos. Um evento **data** sempre está relacionado à recepção de dados, enquanto **end** estabelece o término de uma comunicação, diferentemente de **close**, que indica o fechamento de uma conexão ou outro recurso.

Caso você tenha que ler um arquivo grande, pode ser utilizado o modelo de eventos, no qual a cada evento **data** uma parte do arquivo é recebida e tratada.

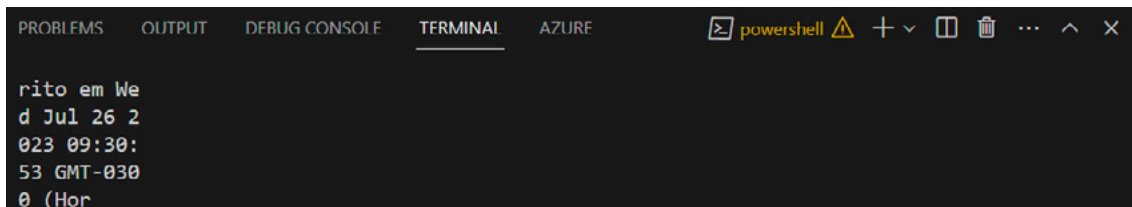
```
import $fs from 'fs';

const reader = $fs.createReadStream('log.txt', {
  flag: 'r', encoding: 'UTF-8',
  start: 5, end: 200, highWaterMark: 10
});

reader.on('data', (chunk) => { console.log(chunk); });
```

No exemplo apresentado, criamos um fluxo (**stream**) de leitura, configurado para iniciar na posição 5 do arquivo e ler até a posição 200, em blocos contendo 10 caracteres. Através do evento data, cada fragmento (**chunk**) lido é impresso no console, como na figura seguinte.

Figura 1: Leitura de arquivo via stream



Fonte: Elaboração própria.

Agora, observe o servidor definido no fragmento de código seguinte.

```
const server = http.createServer((req, res) =>{
  res.write("A");
  res.write("B");
  res.end("C");
}).on("close", ()=>console.log("Servidor encerrado"));
```

No servidor de exemplo, temos uma callback que envia os dados em três partes para o cliente, sendo as duas primeiras via **write**, e a terceira via **end**, que também finaliza a comunicação. Além disso, foi definida uma callback para o evento **close**, avisando, através do console, quando o servidor é finalizado.

Uma requisição para o servidor seria definida como no código seguinte.

```
const req = http.request(  
  {hostname: 'localhost', port: 3000, method: 'GET'},  
  (res) => {  
    res.on('data', (d) => console.log("Resp: "+d.toString()));  
    res.on('end', () => console.log("Fim da comunicação"));  
    res.on('close', () => console.log("Conexão encerrada"));  
  });  
req.end();
```

Note que precisamos do evento **data**, para apresentar as letras recebidas no console, do evento **end**, indicando o fim da comunicação, e do evento **close**, que ocorrerá com o término da conexão.

Considerações Finais da Aula

A programação assíncrona é uma necessidade cada vez mais comum em qualquer ambiente de desenvolvimento atual, e não seria diferente na plataforma NodeJS. Por lidar com o JavaScript, esse modelo de programação é utilizado de forma simples, com base em promises e funções da callback, além de um conjunto de palavras reservadas específico para essa área da programação.

Após analisar o modelo de programação do Javascript, pudemos utilizar as bibliotecas para gerenciamento de arquivos e conexões remotas com elementos assíncronos, garantindo que nossos sistemas executem de forma fluida e com maior eficiência.

Por fim, pudemos compreender o uso de eventos do NodeJS na prática e vimos como ocorre a integração com o event loop, associando cada evento a uma função de callback. Esse estilo de programação é crucial para as operações de entrada e saída, como vimos no gerenciamento de arquivos e na recepção de dados nas conexões HTTP.

Materiais Complementares

Artigo

Asynchronous flow control

2023, Rex f, Claudio W e Nazaré da Piedade. NodeJS.

O artigo traz uma descrição completa do modelo de programação assíncrona no NodeJs, com diversos tópicos disponibilizados pela navegação lateral, e oferece tradução para o português, embora seja o de Portugal.

Link para acesso: <https://nodejs.dev/en/learn/asynchronous-flow-control/> (acesso em 12 set. 2023.)

Artigo

NodeJS File System Module

2023, W3C. NodeJS.

O artigo, que é parte da documentação oferecida pelo site NodeJS Tutorial, da W3C, explica a utilização do módulo fs, para gerenciamento de arquivos, com diversos exemplos no modelo assíncrono.

Link para acesso: https://www.w3schools.com/nodejs/nodejs_filesystem.asp (acesso em 12 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidadePaginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 12 set. 2023.